컴퓨터 비전

4주차

목차

- **영상 파일 처리**
  - 영상 파일 읽기
  - 영상 파일 저장
- **동영상 파일 처리**
  - 영상 파일 읽기
  - 영상 파일 저장
- **기본 배열 처리 함수**
  - 채널 처리 함수
  - 영상 채널 처리
- **과제1**
- **과제2**

이미지 파일 처리

- **이미지 파일 처리 주요 함수**

- function : cv.imread(filename[, flags]) ->retval
  - 파일에서 이미지를 읽어오는 함수
  - Parameters
    - filename : 읽어올 파일의 이름
    - Flags : 색상 유형을 결정하는 flag (cv::ImreadModes)

- function : cv.imwrite(filename, img[, params]) ->retval
  - 이미지를 특정 파일에 저장하는 함수
  - Parameters
    - filename : 저장할 파일의 이름
    - Img : 저장하고자 하는 이미지
    - Flags : 압축 방식을 결정하는 flag (cv::ImwriteFlags)

영상 파일 처리

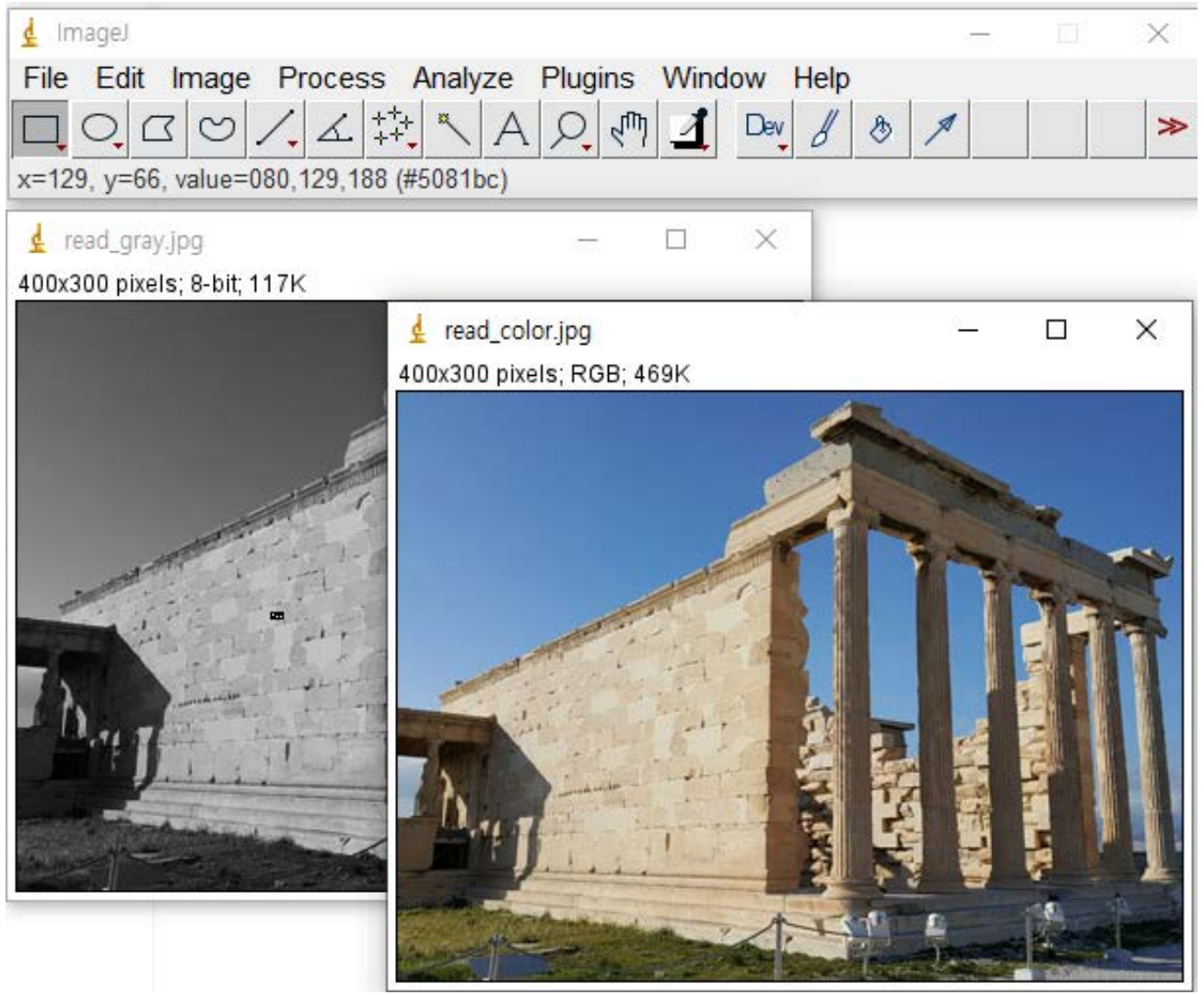
- **색상 유형 결정 상수**

〈표 4.4.1〉 행렬의 컬러 타입 결정 상수

옵션	값	설명
cv2.IMREAD_UNCHANGED	-1	입력 파일에 정의된 타입의 영상을 그대로 반환(알파(alpha) 채널 포함)
cv2.IMREAD_GRAYSCALE	0	명암도(grayscale) 영상으로 변환하여 반환
cv2.IMREAD_COLOR	1	컬러 영상으로 변환하여 반환
cv2.IMREAD_ANYDEPTH	2	입력 파일에 정의된 깊이(depth)에 따라 16비트/32비트 영상으로 변환, 설정되지 않으면 8비트 영상으로 변환
cv2.IMREAD_ANYCOLOR	4	입력 파일에 정의된 타입의 영상을 반환

영상 파일 처리

- **Image 처리 도구**
  - https://imagej.net/ij/download.html
  - java 기반 이미지 처리/분석 도구
  - 무료 라이센스



영상 파일 처리

- **영상 파일 읽기 코드**
  - read_gray.jpg 파일 필요
  - [학습정보]->[학습자료] 페이지 다운로드
  - .py파일과 동일 위치 저장

```
def print_matInfo(name, image):
    if image.dtype == 'uint8': mat_type = 'CV_8U'
    elif image.dtype == 'int8': mat_type = 'CV_8S'
    elif image.dtype == 'uint16': mat_type = 'CV_16U'
    elif image.dtype == 'int16': mat_type = 'CV_16S'
    elif image.dtype == 'float32': mat_type = 'CV_32F'
    elif image.dtype == 'float64': mat_type = 'CV_64F'
    nchannel = 3 if image.ndim == 3 else 1

    print("%12s: depth(%s), channels(%s) -> mat_type(%sC%d)"
          % (name, image.dtype, nchannel, mat_type, nchannel))


title1 = 'gray2gray'
title2 = 'gray2color'
```

영상 파일 처리

- 영상 파일 읽기 코드

```
gray2gray = cv2.imread("read_gray.jpg", cv2.IMREAD_GRAYSCALE)
gray2color = cv2.imread("read_gray.jpg", cv2.IMREAD_COLOR)

if gray2gray is None or gray2color is None:
    raise Exception("read Error")

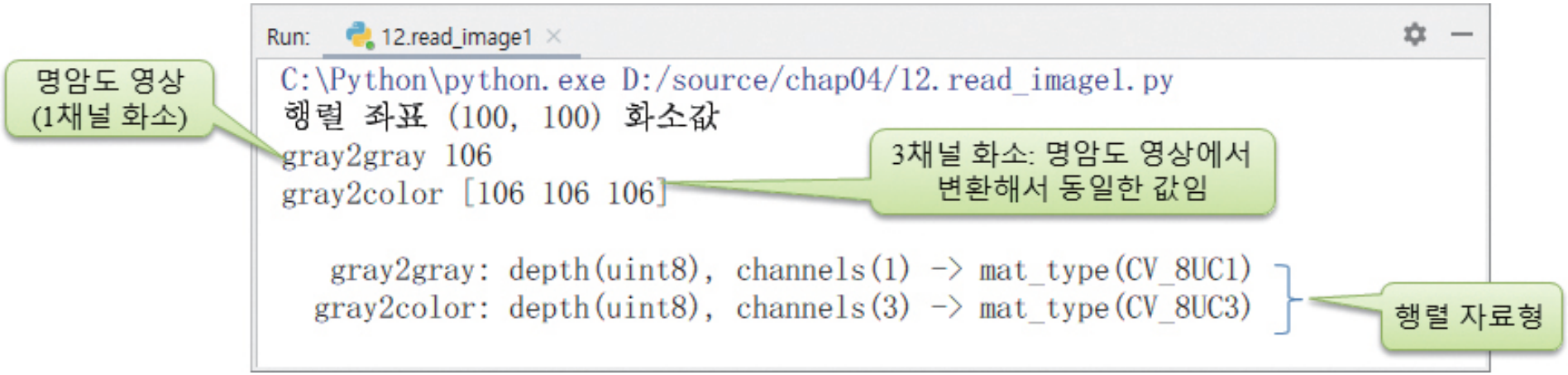
print("pixel value : [100, 100]")
print("title1 : ", gray2gray[100,100])
print("title2 : ", gray2color[100,100])

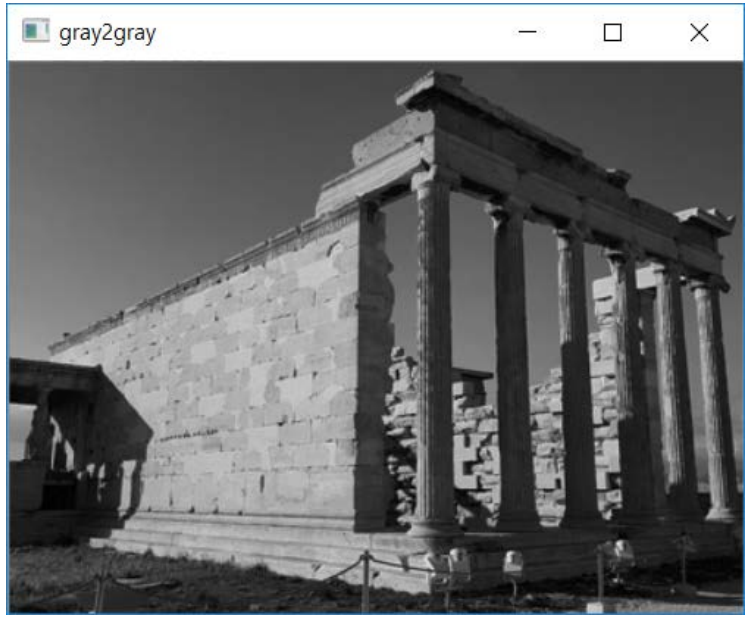
print_matInfo(title1, gray2gray)
print_matInfo(title2, gray2color)

cv2.imshow(title1, gray2gray)
cv2.imshow(title2, gray2color)
cv2.waitKey(0)
```

영상 파일 처리

- 영상 파일 읽기 코드 결과







영상 파일 처리

- **컬러 영상 파일 읽기**
  - read_color.jpg 파일 필요
  - [학습정보]->[학습자료] 페이지 다운로드
  - .py파일과 동일 위치 저장

```python
import cv2
from Common.utils import print_matInfo

title1 = 'color2gray'
title2 = 'color2color'
color2gray = cv2.imread("read_color.jpg", cv2.IMREAD_GRAYSCALE)
color2color = cv2.imread("read_color.jpg", cv2.IMREAD_COLOR)

if color2gray is None or color2color is None:
    raise Exception("read Error")
print("pixel value : [100, 100]")
print("title1 : ", color2gray[100,100])
print("title2 : ", color2color[100,100])

print_matInfo(title1, color2gray)
print_matInfo(title2, color2color)
```

영상 파일 처리

- **실행결과**

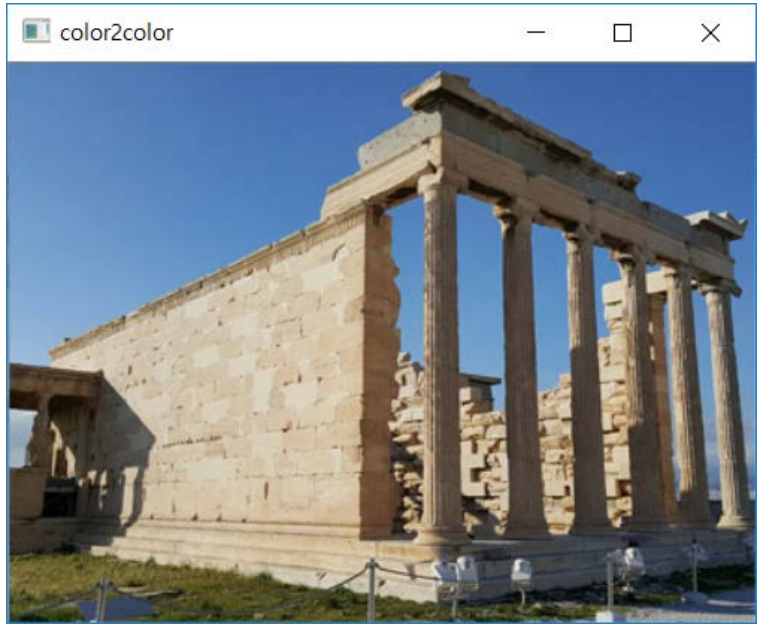
컬러 영상도 명암도 타입으로 읽으면 1채널 영상이 됨

```
Run:  13.read_image2
C:\Python\python.exe D:/source/chap04/13.read_image2.py
행렬 좌표 (100, 100) 화소값
color2gray 137
color2color [197 145  98]

  color2gray: depth(uint8), channels(1) -> mat_type(CV_8UC1)
 color2color: depth(uint8), channels(3) -> mat_type(CV_8UC3)
```

3채널 컬러 영상 화소값





영상 파일 처리

- **16/32 uchar 영상 파일 읽기**
  - read_16.jpg 파일 필요
  - read_32.jpg 파일 필요
  - [학습정보]->[학습자료] 페이지 다운로드
  - .py파일과 동일 위치 저장

```
title1 = '16bit unchanged'
title2 = '32bit unchanged'
color2unchanged1 = cv2.imread("read_16.tif", cv2.IMREAD_UNCHANGED)
color2unchanged2 = cv2.imread("read_32.tif", cv2.IMREAD_UNCHANGED)

if color2unchanged1 is None or color2unchanged2 is None:
    raise Exception("read Error")
print("pixel value : [10, 10]")
print(title1, ":", type(color2unchanged1[10][10][0]), color2unchanged1[10,10])
print(title2, ":", type(color2unchanged2[10][10][0]), color2unchanged2[10,10])

print_matInfo(title1, color2unchanged1)
print_matInfo(title2, color2unchanged2)

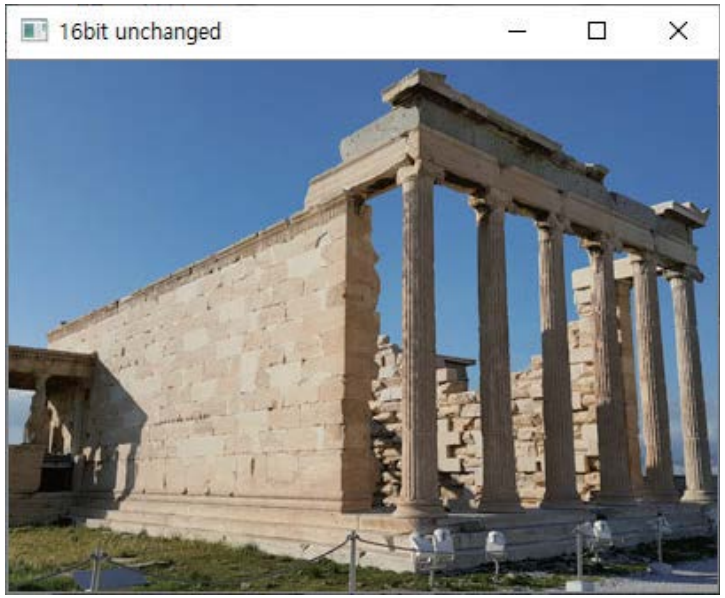
cv2.imshow(title1, color2unchanged1)
cv2.imshow(title2, color2unchanged2)
cv2.waitKey(0)
```

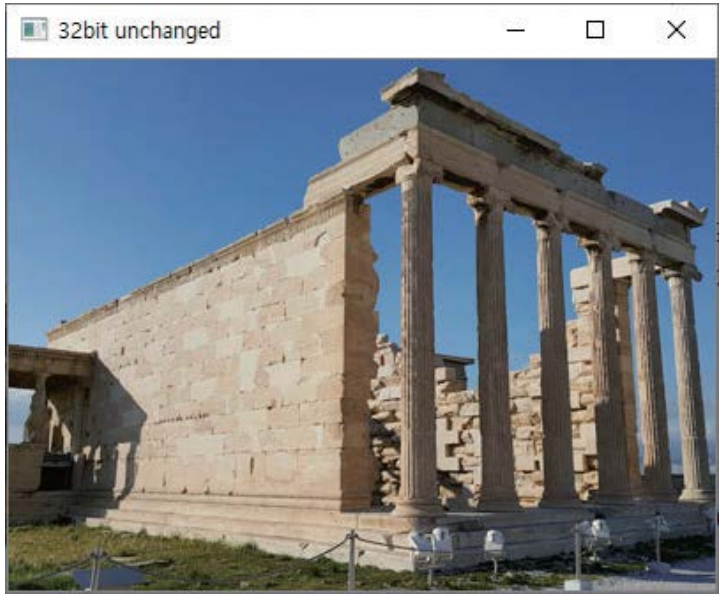
영상 파일 처리

- 실행결과

```
Run:  14.read_image3
C:\Python\python.exe D:/source/chap04/14.read_image3.py
16/32비트 영상 행렬 좌표 (10, 10) 화소값
16bit unchanged 원소 자료형  <class 'numpy.uint16'>
16bit unchanged 화소값(3원소)  [48573 34438 23387]
32bit unchanged 원소 자료형  <class 'numpy.float32'>
32bit unchanged 화소값(3원소)  [0.7456989  0.52237624 0.36376417]

16bit unchanged: depth(uint16), channels(3) -> mat_type(CV_16UC3)
32bit unchanged: depth(float32), channels(3) -> mat_type(CV_32FC3)
```





영상 파일 저장

- **cv2.imwrite() 함수**
  - 행렬을 영상파일로 저장
  - 확장자에 따라서 JPG, BMP, PNG, TIF, PPM 등의 영상파일 포맷 저장 가능

〈표 4.4.3〉 압축 방식에 사용되는 params 인수 튜플(paramId, paramValue)의 예시

paramId	paramValue (기본값)	설명
cv2.IMWRITE_JPEG_QUALITY	0~100 (95)	JPG 파일 화질, 높은 값일수록 화질 좋음
cv2.IMWRITE_PNG_COMPRESSION	0~9 (3)	PNG 파일 압축 레벨, 높은 값일수록 용량은 적어지고, 압축 시간이 길어짐
cv2.IMWRITE_PXM_BINARY	0 or 1 (1)	PPM, PGM 파일의 이진 포맷 설정

영상 파일 저장

- 영상 파일 저장 코드

```python
import cv2

image = cv2.imread("read_color.jpg", cv2.IMREAD_COLOR)

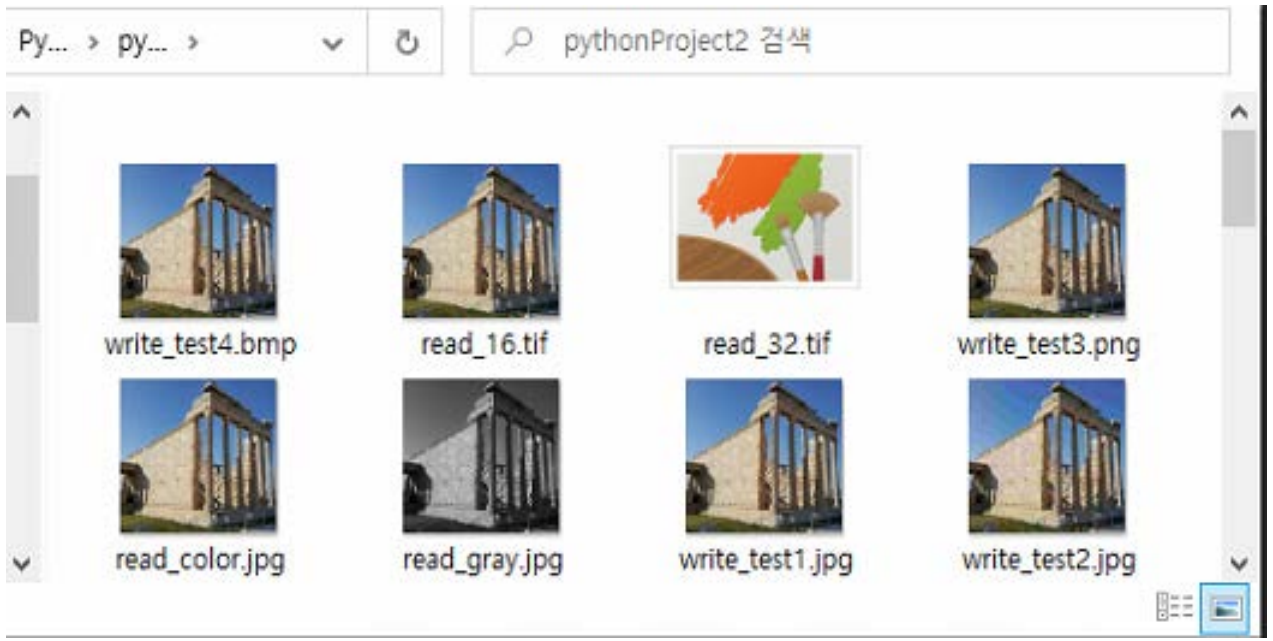
if image is None:
    raise Exception("read Error")

params_jpg = (cv2.IMWRITE_JPEG_QUALITY, 10)
params_png = (cv2.IMWRITE_PNG_COMPRESSION, 9)

cv2.imwrite("write_test1.jpg", image)
cv2.imwrite("write_test2.jpg", image, params_jpg)
cv2.imwrite("write_test3.png", image, params_png)
cv2.imwrite("write_test4.bmp", image)
print('End')
```

영상 파일 저장

- **실행결과**
  - 크기 및 영상 형식 확인



동영상 파일 처리

- **동영상 파일**
  - 초당 30프레임 저장, 압축 필요 시 → 압축 코덱(codec) 사용함
- **OpenCV는 동영상을 처리할 수 있는 클래스 제공**
  - 단 전문가용 동영상 처리 프로그램보다 지원 기능이 적음
  - 간단한 작업 이외에는 전문 프로그램을 사용하는 것을 추천

VideoCapture 클래스 함수 설명		
cv2.VideoCapture() → 〈VideoCapture object〉 cv2.VideoCapture(filename) → 〈VideoCapture object〉 cv2.VideoCapture(device) → 〈VideoCapture object〉 ■ 설명: 생성자, 3가지 VideoCapture 객체 선언 방법을 지원한다.		
인수 설명	■ filename ■ device	개방할 동영상파일의 이름 혹은 영상 시퀀스 개방할 동영상 캡처 장치의 ID(카메라 한대만 연결되면 0을 지정)
cv2.VideoCapture.open(filename) → retval cv2.VideoCapture.open(device) → retval ■ 설명: 동영상 캡처를 위한 동영상파일 혹은 캡처 장치를 개방한다.		
cv2.VideoCapture.isOpened() → retval ■ 설명: 캡처 장치의 연결 여부를 반환한다.		
cv2.VideoCapture.release() → None ■ 설명: 동영상파일이나 캡처 장치를 해제한다. (클래스 소멸자에 의해서 자동으로 호출되므로 명시적으로 수행하지 않아도 됨)		

동영상 파일 처리

- **OpenCV 동영상 처리 클래스**

cv2.VideoCapture.get(propId) → retval

■ 설명: 비디오 캡처의 속성 식별자로 지정된 속성의 값을 반환한다. 캡처 장치가 제공하지 않는 속성은 0을 반환한다.

인수 설명	■ propId	속성 식별자 – <표 4.5.1>에서 정리	

cv2.VideoCapture.set(propId, value) → retval

■ 설명: 지정된 속성 식별자로 비디오 캡처의 속성을 설정한다.

인수 설명	■ propId	속성 식별자
	■ value	속성 값

cv2.VideoCapture.grab() → retval

■ 설명: 캡처 장치나 동영상파일에서 다음 프레임을 잡는다.

retrieve([, image[, flag]]) → retval, image

■ 설명: grab()으로 잡은 프레임을 디코드해서 image 행렬로 전달한다.

인수 설명	■ image	잡은 프레임이 저장되는 행렬
	■ flag	프레임 인덱스

cv2.VideoCapture.read([image]) → retval, image

■ 설명: 캡처 장치나 동영상파일에서 다음 프레임을 잡아 디코드해서 image 행렬로 전달한다.

동영상 파일 읽기

- **동영상 파일 읽기 코드**
  - video_file.avi 파일 필요
  - video_file2.avi 파일 필요
  - [학습정보]->[학습자료] 페이지 다운로드
  - .py파일과 동일 위치 저장

```
capture = cv2.VideoCapture("video_file2.avi")

if not capture.isOpened(): raise Exception("Unable to open video file.")

frame_rate = capture.get(cv2.CAP_PROP_FPS)
delay = int(1000 / frame_rate)

while True:
    ret, frame = capture.read()

    if not ret or cv2.waitKey(delay) >= 0:
        break

    cv2.imshow("Video file", frame)

capture.release()
```

동영상 파일 읽기

- **동영상 파일 속성 확인 코드**
  - video_file.avi 파일 필요
  - video_file2.avi 파일 필요
  - [학습정보]->[학습자료] 페이지 다운로드
  - .py파일과 동일 위치 저장

- **frame_rate : 초당 프레임 수**
- **delay : 프레임당 시간**
- **width : 너비**
- **height : 높이**

```
resolution: 640.0 x 360.0
frame_rate: 29.97
delay: 33
```

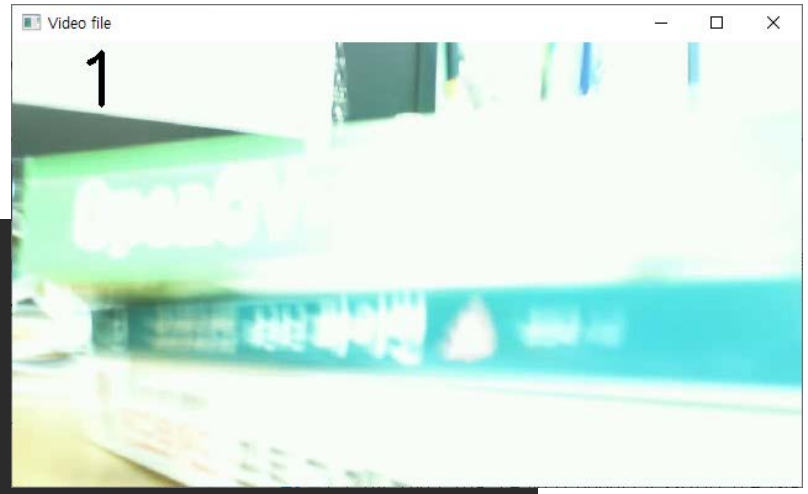
```python
frame_rate = capture.get(cv2.CAP_PROP_FPS)
delay = int(1000 / frame_rate)

width = capture.get(cv2.CAP_PROP_FRAME_WIDTH)
height = capture.get(cv2.CAP_PROP_FRAME_HEIGHT)
print('resolution:', width, 'x', height)
print("frame_rate:", frame_rate)
print("delay:", delay)
frame_cnt = 0
font = cv2.FONT_HERSHEY_SIMPLEX

while True:
```

동영상 파일 읽기

- frame 번호 출력




```
frame_cnt = 0
font = cv2.FONT_HERSHEY_SIMPLEX

while True:
    ret, frame = capture.read()

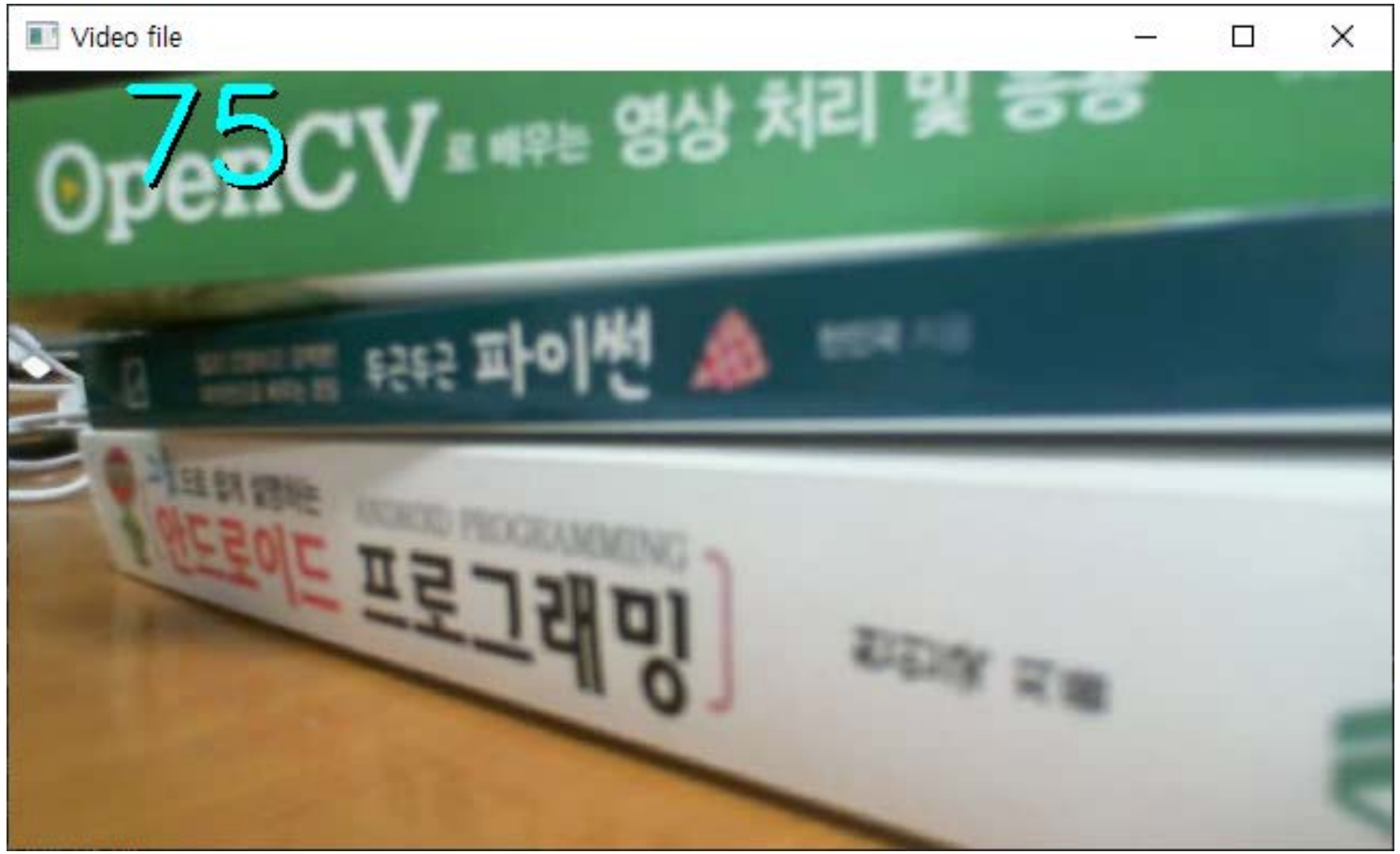
    if not ret or cv2.waitKey(delay) >= 0:
        break

    frame_cnt += 1
    cv2.putText(frame, str(frame_cnt), (50, 50), font, 2, (0,0,0), 3)
    cv2.imshow("Video file", frame)

capture.release()
```

동영상 파일 읽기

- **실습**
  - putText를 사용하여 frame 번호를 그림자 효과를 이용하여 출력



동영상 파일 저장

- **OpenCV 동영상 저장 클래스**

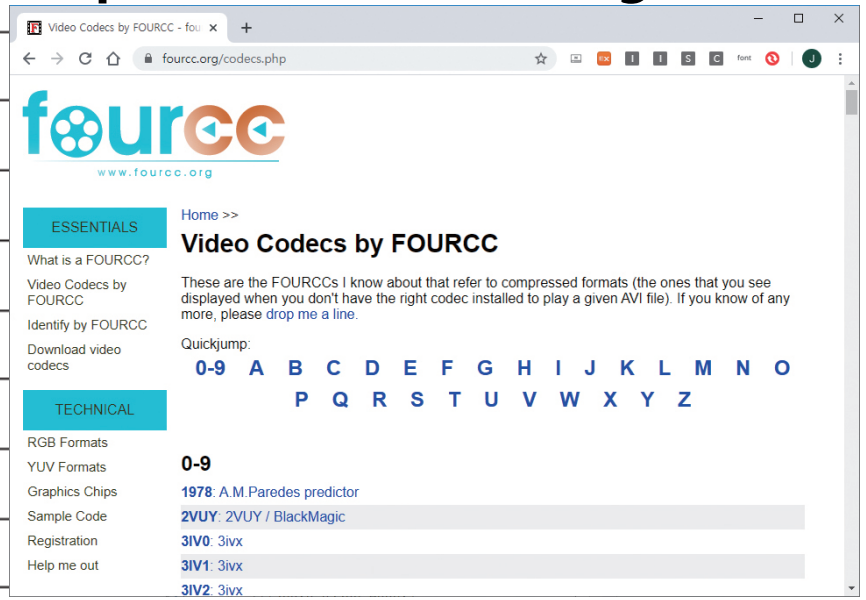
VideoWriter 클래스 설명		
cv2.VideoWriter([filename, fourcc, fps, frameSize[, isColor]]) → <VideoWriter object> cv2.VideoWriter.open(filename, fourcc, fps, frameSize[, isColor]) → retval		
인수 설명	■ filename	출력 동영상파일의 이름
	■ fourcc	프레임 압축에 사용되는 코덱의 4-문자
	■ fps	생성된 동영상 프레임들의 프레임률(초당 프레임수)
	■ frameSize	동영상 프레임의 크기(가로×세로)
	■ isColor	True면 컬러 프레임으로 인코딩, False면 명암도 프레임으로 인코딩
cv2.VideoWriter.isOpened() → retval ■ 설명: 캡처 장치나 동영상파일이 열려있는지 확인한다.		
cv2.VideoWriter.write(image) → None ■ 설명: image 프레임을 파일로 저장한다.		
cv2.VideoWriter.open() ■ 설명: 영상을 동영상파일의 프레임으로 저장하기 위해 동영상파일을 개방한다. 인수는 생성자의 인수와 동일하다.		
cv2.VideoWriter.isOpened() ■ 설명: 동영상파일 저장을 위해 VideoWriter 객체의 개방 여부를 확인한다.		

동영상 파일 저장

〈표 4.5.1〉 카메라의 주요 속성 식별자

속성 상수	설명
cv2.CAP_PROP_POS_MSEC	동영상파일의 현재 위치인 밀리초(millisecond)
cv2.CAP_PROP_POS_FRAMES	캡처되는 프레임의 번호
cv2.CAP_PROP_POS_AVI_RATIO	동영상파일의 상대적 위치 (0 – 시작, 1 – 끝)
cv2.CAP_PROP_FRAME_WIDTH	프레임의 너비
cv2.CAP_PROP_FRAME_HEIGHT	프레임의 높이
cv2.CAP_PROP_FPS	초당 프레임 수
cv2.CAP_PROP_FOURCC	코덱의 4문자
cv2.CAP_PROP_FRAME_COUNT	동영상파일의 총 프레임 수
cv2.CAP_PROP_FORMAT	cv2.VideoCatpure.retrieve()이 반환하는 행렬 포맷
cv2.CAP_PROP_BRIGHTNESS	카메라에서 영상의 밝기
cv2.CAP_PROP_CONTRAST	카메라에서 영상의 대비
cv2.CAP_PROP_SATURATION	카메라에서 영상의 포화도
cv2.CAP_PROP_HUE	카메라에서 영상의 색상
cv2.CAP_PROP_GAIN	카메라에서 영상의 Gain
cv2.CAP_PROP_EXPOSURE	카메라에서 노출
cv2.CAP_PROP_AUTOFOCUS	자동 초점 조절

https://www.fourcc.org/codecs.php



동영상 파일 저장

〈표 4.5.2〉 주요 코덱 문자

속성 상수	설명
cv2.VideoWriter_fourcc('D', 'I', 'V', '4') cv2.VideoWrite_fourcc(*'DIV4')	DivX MPEG-4
cv2.VideoWriter_fourcc('D', 'I', 'V', '5') cv2.VideoWrite fourcc(*'DIV5')	Div5
cv2.VideoWriter_fourcc('D', 'I', 'V', 'X') cv2.VideoWrite_fourcc(*'DIVX')	DivX
cv2.VideoWriter_fourcc('D', 'X', '5', '0') cv2.VideoWrite_fourcc(*'DX50')	DivX MPEG-4
cv2.VideoWriter_fourcc('F', 'M', 'P', '4') cv2.VideoWrite_fourcc(*'FMP4')	FFMpeg
cv2.VideoWriter_fourcc('I', 'Y', 'U', 'V') cv2.VideoWrite_fourcc(*'TYUV')	IYUV
cv2.VideoWriter_fourcc('M', 'J', 'P', 'G') cv2.VideoWrite_fourcc(*'MJPG')	Motion JPEG codec
cv2.VideoWriter_fourcc'M', 'P', '4', '2') cv2.VideoWrite_fourcc(*'MP42')	MPEG4 v2
cv2.VideoWriter_fourcc('M', 'P', 'E', 'G') cv2.VideoWrite_fourcc(*'MPEG')	MPEG codecs
cv2.VideoWriter_fourcc('X', 'V', 'I', 'D') cv2.VideoWrite_fourcc(*'XVID')	XVID codecs
cv2.VideoWriter_fourcc('X', '2', '6', '4') cv2.VideoWrite_fourcc(*'X264')	H.264/AVC codecs
-1	코덱 선택 대화상자 띄움

동영상 파일 저장

- **동영상 파일 저장 코드**
  - 원본 코드에서 추가

```
frame_rate = capture.get(cv2.CAP_PROP_FPS)
delay = int(1000 / frame_rate)

width = int(capture.get(cv2.CAP_PROP_FRAME_WIDTH))
height = int(capture.get(cv2.CAP_PROP_FRAME_HEIGHT))
print('resolution:', width, 'x' height)
frame_cnt = 0
font = cv2.FONT_HERSHEY_SIMPLEX

fourcc = cv2.VideoWriter_fourcc(*'FMP4')
writer = cv2.VideoWriter("video_file2_out.mp4", fourcc, frame_rate, (width, height))
```

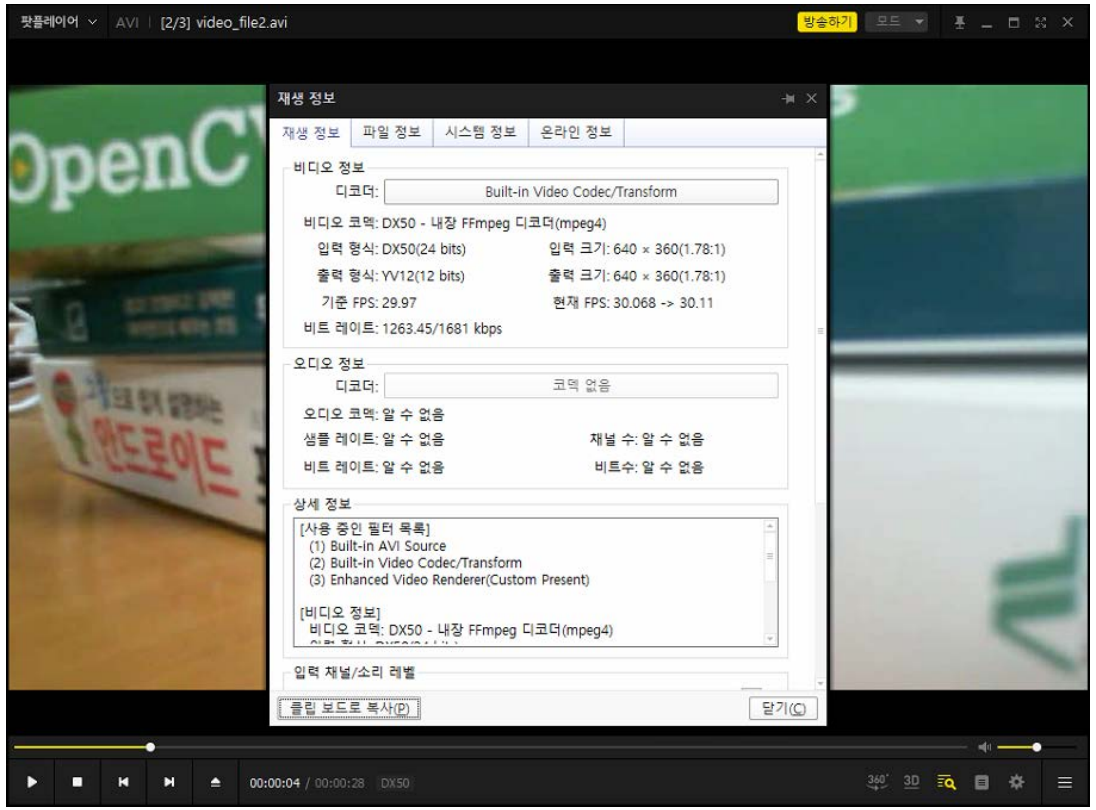
```
    writer.write(frame)
    cv2.imshow("Video file", frame)

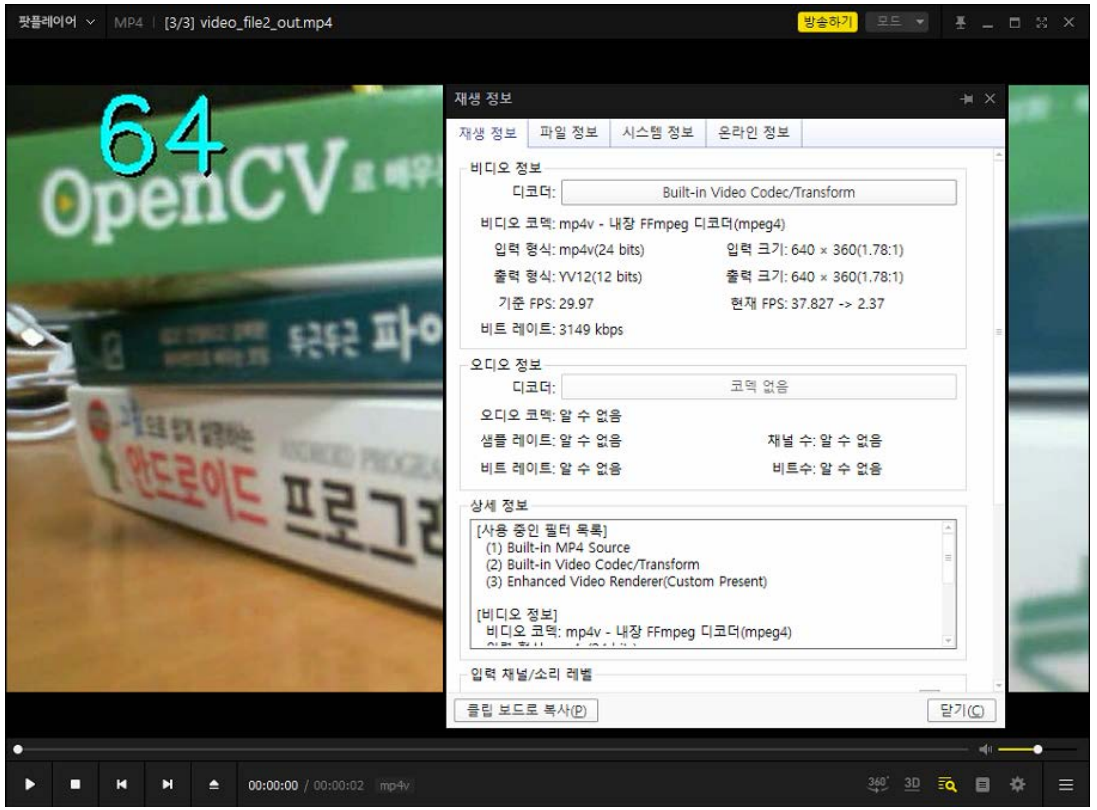


capture.release()
writer.release()
```

동영상 파일 저장

- **동영상 플레이어**
  - 원본 영상에서 새로운 영상으로 변경된 정보 확인 가능
  - (팟플레이어를 영상 플레이어로 사용)





기본 배열(Array) 처리 함수

- **파이썬에서 배열**
  - 1차원 데이터 : 벡터
  - 2차원 데이터 : 행렬
  - 1차원과 2차원 데이터 통칭 : 배열

함수 설명		
cv2.flip(src, flipCode[, dst]) → dst ■설명: 입력된 2차원 배열을 수직, 수평, 양축으로 뒤집는다.		
인수 설명	■src , dst ■flipCode - 0 : x축을 기준으로 위아래로 뒤집는다. - 1 : y축을 기준으로 좌우로 뒤집는다. - -1 : 양축(x축, y축 모두)을 기준으로 뒤집는다.	입력 배열, 출력 배열 배열을 뒤집는 축
cv2.repeat(src, ny, nx[, dst]) → dst ■설명: 입력 배열의 반복된 복사본으로 출력 배열을 채운다.		
인수 설명	■src, dst ■ny, nx	입력 배열, 출력 배열 수직 방향 , 수평방향 반복 횟수
cv2.transpose(src[, dst]) → dst ■설명: 입력 행렬의 전치 행렬을 출력으로 반환한다.		
인수 설명	■src, dst	입력 배열, 출력 배열

기본 배열(Array) 처리 함수

- 배열 처리 실행 코드

```
image = cv2.imread("flip_test.jpg", cv2.IMREAD_COLOR)
if image is None: raise Exception("Unable to open image file.")

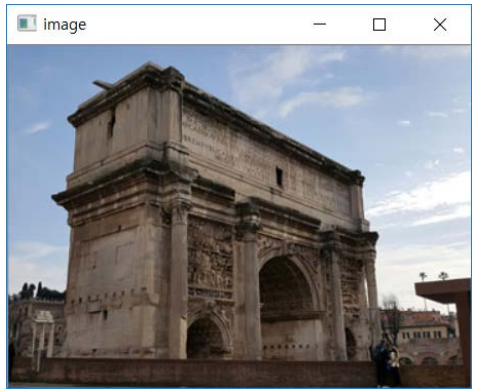
x_axis = cv2.flip(image, 0)
y_axis = cv2.flip(image, 1)
xy_axis = cv2.flip(image, -1)
rep_image = cv2.repeat(image, 1, 2)
trans_image = cv2.transpose(image)

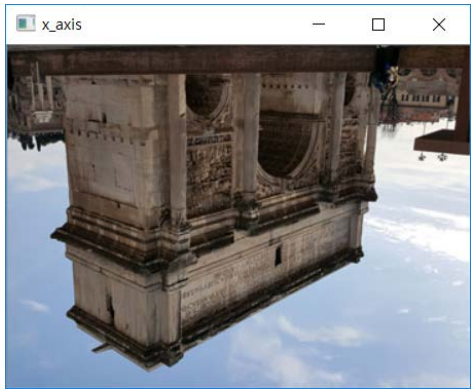
titles = ['image', 'x_axis', 'y_axis', 'xy_axis', 'rep_image', 'trans_image']

for title in titles:
    cv2.imshow(title, eval(title))
cv2.waitKey(0)
```

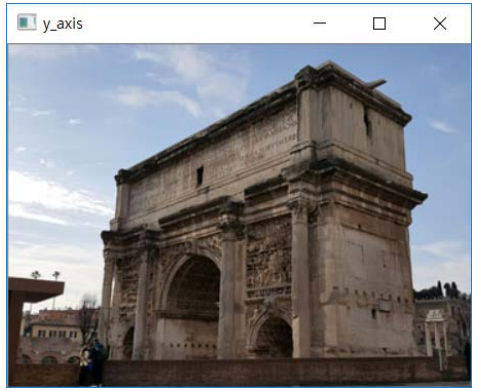
기본 배열(Array) 처리 함수

- 실행 결과







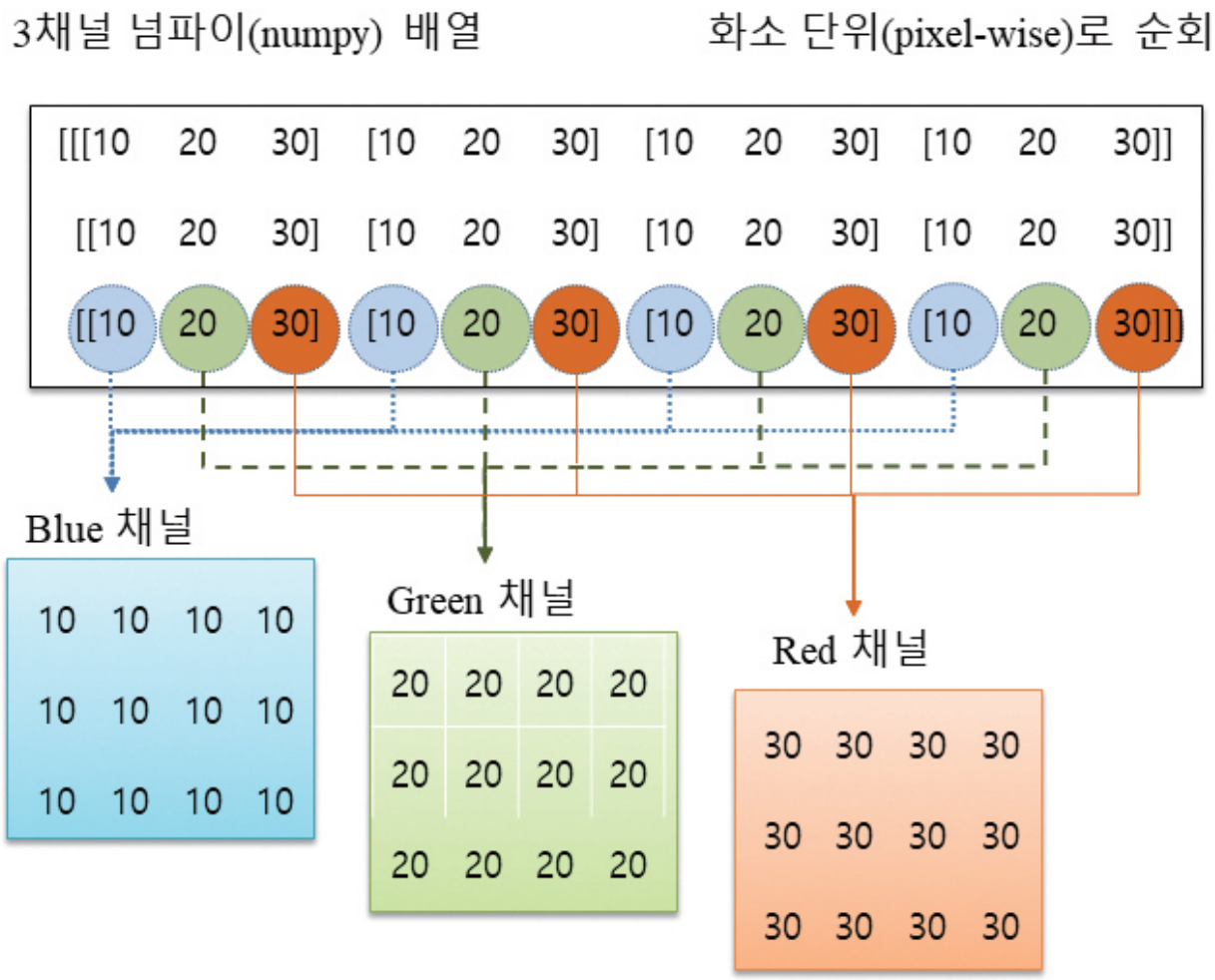






채널 처리 함수

- 채널의 개념



채널 처리 함수

- **채널 처리 함수**

함수 설명		
cv2.merge(mv[, dst]) → dst ■ 설명: 여러 개의 단일채널 배열을 다채널 배열로 합성한다.		
인수 설명	■ mv ■ dst	합성될 입력 배열 혹은 벡터, 합성될 단일채널 배열들의 크기와 깊이(depth)가 동일해야 함 입력 배열과 같은 크기와 같은 깊이의 출력 배열
cv2.split(m[, mv]) → mv ■ 설명: 다채널 배열을 여러 개의 단일채널 배열로 분리한다.		
인수 설명	■ m ■ mv	입력되는 다채널 배열 분리되어 반환되는 단일채널 배열들의 벡터

채널 처리 함수

- 채널 분리 및 합성 코드

```
ch0 = np.zeros((2,4), np.uint8) + 10
ch1 = np.zeros((2,4), np.uint8) + 20
ch2 = np.zeros((2,4), np.uint8) + 30

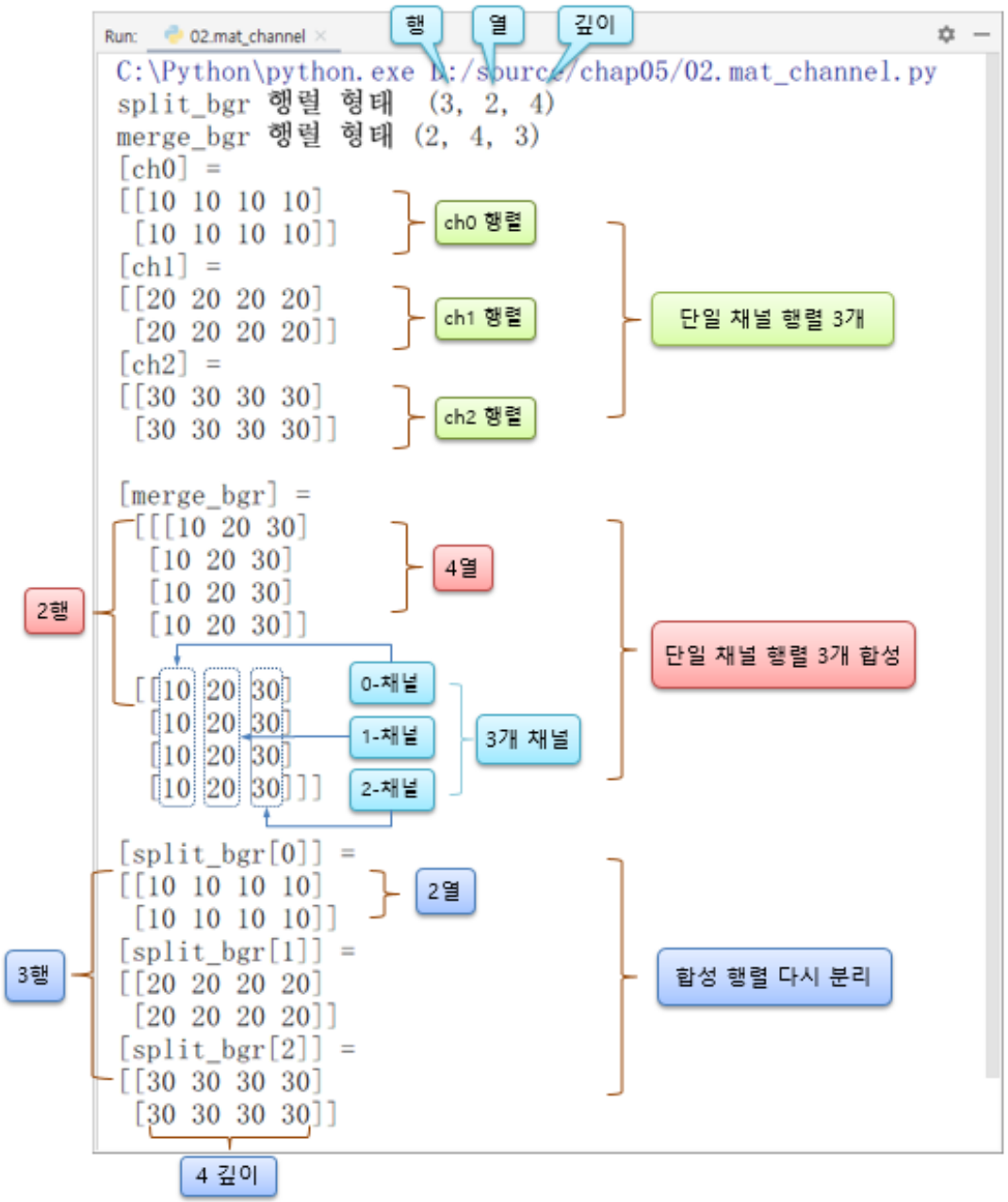
list_bgr = [ch0, ch1, ch2]
merge_bgr = cv2.merge(list_bgr)
split_bgr = cv2.split(merge_bgr)

print("split_bgr Array shape:", np.array(split_bgr).shape)
print("merge_bgr Array shape:", merge_bgr.shape)
print("[ch0] = \n%s" % ch0)
print("[ch1] = \n%s" % ch1)
print("[ch2] = \n%s" % ch2)
print("[merge_bgr] = \n%s \n" % merge_bgr)

print("[split_bgr[0]] = \n%s" % split_bgr[0])
print("[split_bgr[1]] = \n%s" % split_bgr[1])
print("[split_bgr[2]] = \n%s" % split_bgr[2])
```

채널 처리 함수

- 실행 결과


행
열
깊이
Run: 02.mat_channel
C:\Python\python.exe D:/source/chap05/02.mat_channel.py
split_bgr 행렬 형태 (3, 2, 4)
merge_bgr 행렬 형태 (2, 4, 3)
[ch0] =
[[10 10 10 10]
[10 10 10 10]]
[ch1] =
[[20 20 20 20]
[20 20 20 20]]
[ch2] =
[[30 30 30 30]
[30 30 30 30]]
ch0 행렬
ch1 행렬
ch2 행렬
단일 채널 행렬 3개
[merge_bgr] =
[[[10 20 30]
[10 20 30]
[10 20 30]
[10 20 30]]
[[10 20 30]
[10 20 30]
[10 20 30]
[10 20 30]]]
4열
2행
0-채널
1-채널
2-채널
3개 채널
단일 채널 행렬 3개 합성
[split_bgr[0]] =
[[10 10 10 10]
[10 10 10 10]]
[split_bgr[1]] =
[[20 20 20 20]
[20 20 20 20]]
[split_bgr[2]] =
[[30 30 30 30]
[30 30 30 30]]
2열
3행
합성 행렬 다시 분리
4 깊이

영상 채널 처리

- 영상 채널 분리 코드

```
image = cv2.imread("color.jpg", cv2.IMREAD_COLOR)
if image is None: raise Exception("Unable to open image file.")

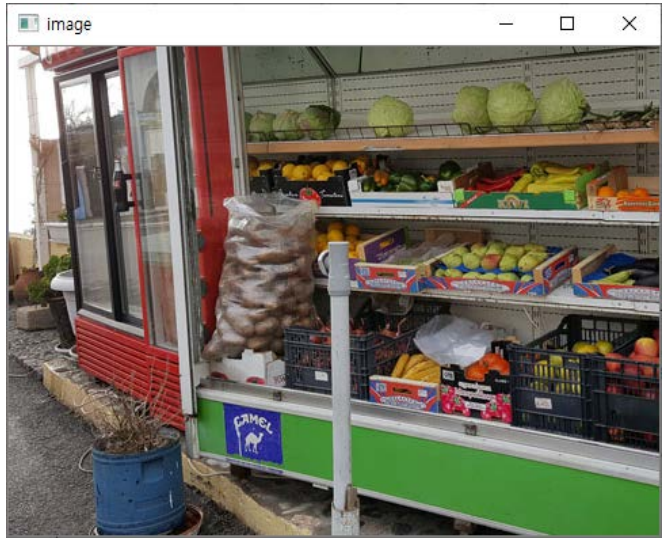
bgr = cv2.split(image)
print("bgr data type:", type(bgr), type(bgr[0]), type(bgr[0][0][0]))
print("bgr element size:", len(bgr))

cv2.imshow("image", image)
cv2.imshow("Blue channel", bgr[0])
cv2.imshow("Green channel", bgr[1])
cv2.imshow("Red channel", bgr[2])

cv2.waitKey(0)
```

영상 채널 처리

- 실행결과

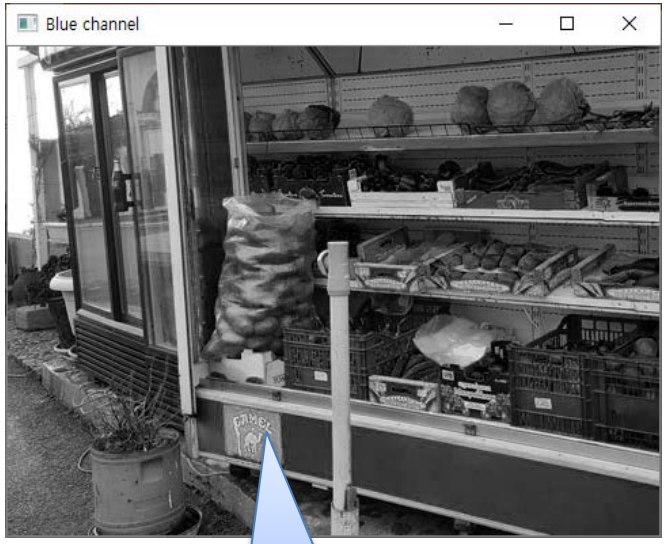



```
bgr data type: <class 'tuple'> <class 'numpy.ndarray'> <class 'numpy.uint8'>
bgr element size: 3
```

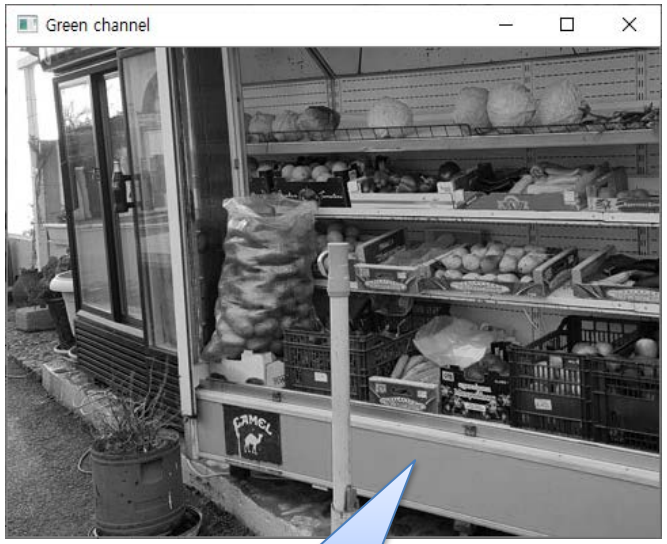
bgr 자료형

bgr 원소(단일채널) 자료형

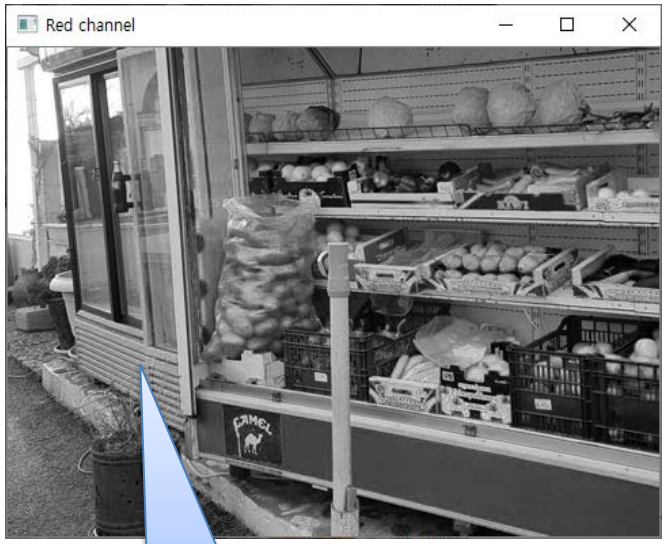
단일채널의 원소 자료형



파란색 - 밝은색



녹색 - 밝은색



붉은색 - 밝은색

영상 채널 처리

- 영상 채널 처리 및 합성

```
image = cv2.imread("color.jpg", cv2.IMREAD_COLOR)
if image is None: raise Exception("Unable to open image file.")

bgr = cv2.split(image)
cv2.add(bgr[0], 100, bgr[0])
merge_bgr = cv2.merge(bgr)
print("bgr data type:", type(bgr), type(bgr[0]), type(bgr[0][0][0]))
print("bgr element size:", len(bgr))

cv2.imshow("image", image)
cv2.imshow("Blue channel", bgr[0])
cv2.imshow("Green channel", bgr[1])
cv2.imshow("Red channel", bgr[2])
cv2.imshow("merge_bgr", merge_bgr)

cv2.waitKey(0)
```

영상 채널 처리

- 영상 채널 처리 및 합성



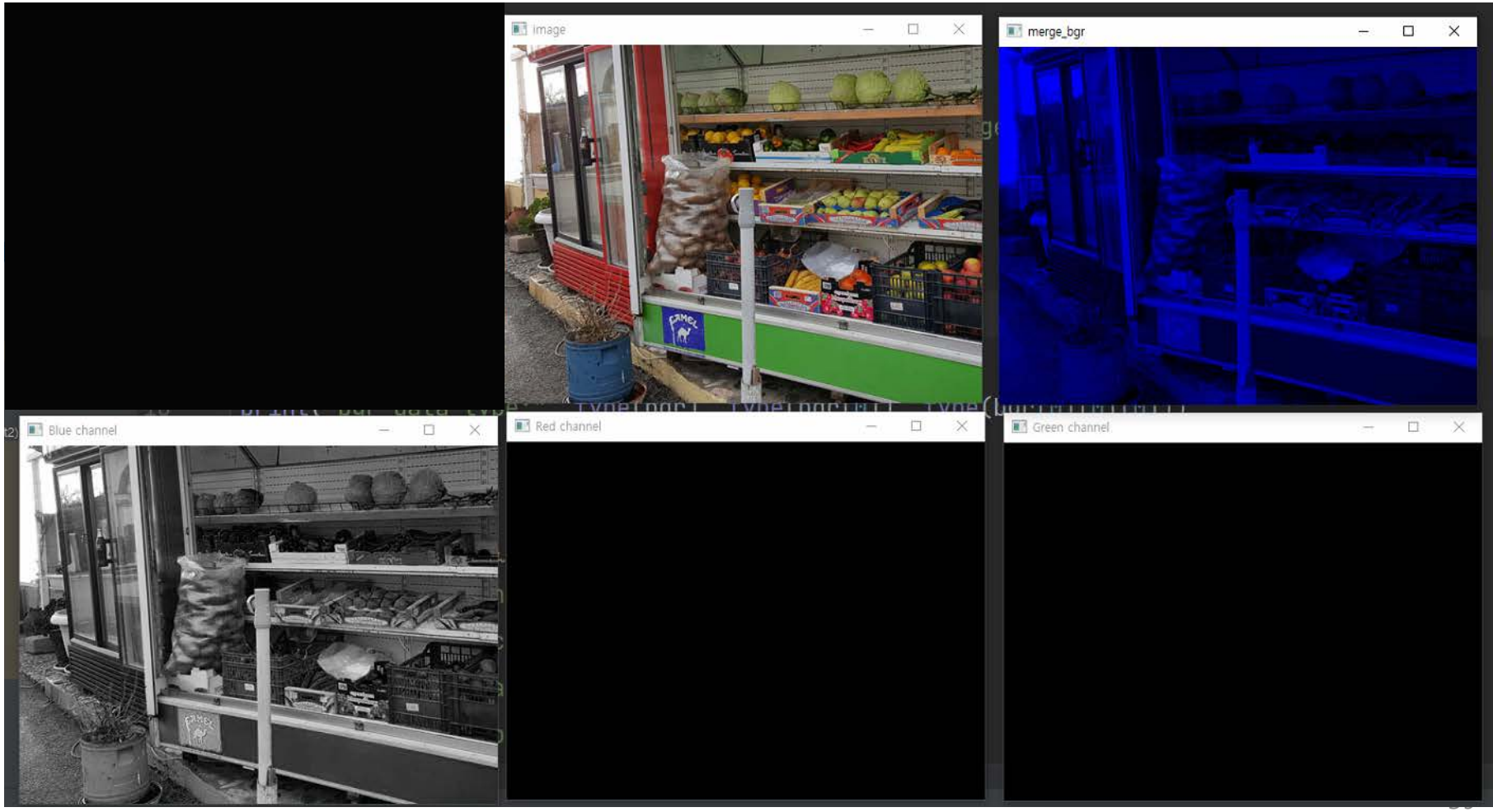
영상 채널 처리

- 영상 채널 처리 및 합성



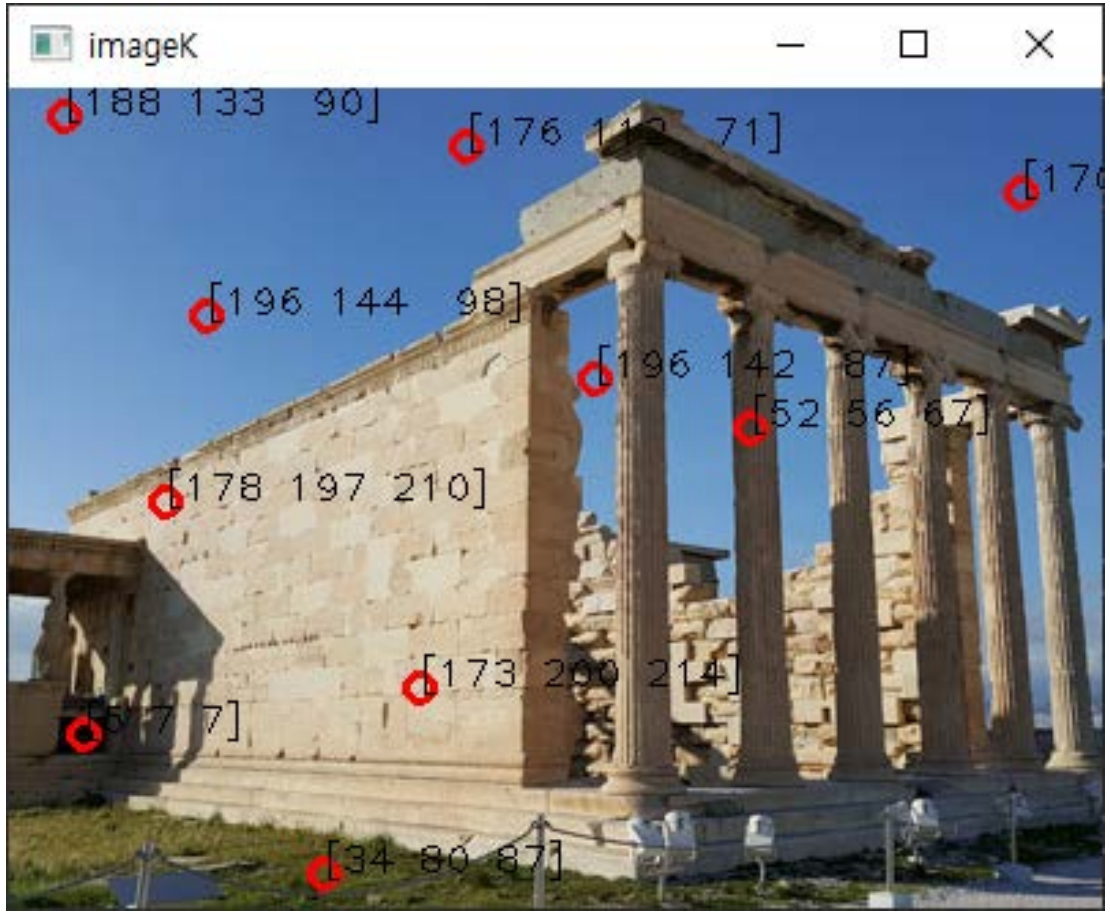
영상 채널 처리

- **영상 채널 처리 및 합성 실습**
  - 우측 결과와 같이 파란색 성분만 남기고 다른 색상의 정보는 제거된 merge_bgr을 만드시오.



과제1

- **이미지 & 이벤트 처리**
  - color 이미지 읽고 띄우기
  - 마우스 좌측 버튼 클릭시
    - 클릭된 위치를 중심으로 원 그리기
    - 그 점의 화소 값을 글씨로 표시



과제1

- **과제 코드 일부**
  - 후단 코드 생략

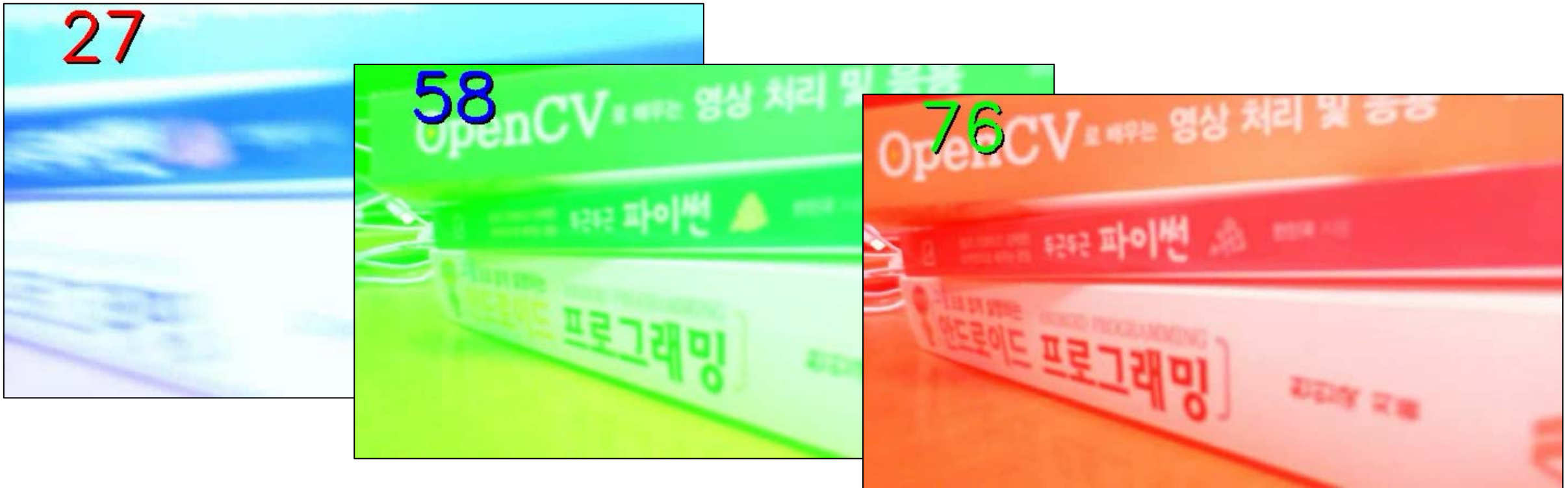
```
def onMouse(event, x, y, flags, param):
    global title
    if event == cv2.[redacted]:
        cv2.circle(image, [redacted])
        cv2.putText([redacted])
        cv2.imshow(title, image)

title = 'imageK'
image = cv2.imread("read_color.jpg", cv2.IMREAD_COLOR)

if image is None:
    raise Exception("read Error")
```

과제2

- **채널 분리 및 글자 쓰기 통합 과제**
  - [1~30 frame] : 파란 화면, 빨간 글씨
  - [31~60 frame] : 녹색 화면, 파란 글씨
  - [61~90 frame] : 빨간 화면, 녹색 글씨
  - 90 frame 이상은 제외
  - video_file2.avi 사용
  - 결과 영상의 저장 코덱 형식은 무관



과제2

- **핵심 코드 중 일부**
  - frame_cnt 변수를 활용하여 구간 분리

```
if not ret or cv2.waitKey(delay) >= 0:
    break
bgr = 
frame_cnt += 1
if frame_cnt > 90:
    break
elif frame_cnt <= 30:
    cv2.
    frame =
    cv2.putText
    cv2.putText
elif frame_cnt <= 60:
    cv2.
    frame =
    cv2.putText
    cv2.putText                                                    3)
elif frame_cnt <= 90:
```